



K-Nearest Neighbors (KNN) — সম্পূর্ণ গাইড

১. KNN কী জিনিস? (What is KNN?)

সহজ বাংলায় আগে বোঝা:

কল্পনা করো তুমি একটা নতুন শহরে গেছো। তোমাকে বলা হলো — "তুমি কোন এলাকায় থাকবে সেটা ঠিক হবে তোমার কাছের ৫ জন প্রতিবেশীর উপর ভিত্তি করে।" মানে, তোমার কাছের বেশিরভাগ প্রতিবেশী যদি ডাক্তার হয়, তাহলে তুমিও ডাক্তার-এলাকায় classified হবে।

KNN ঠিক এই কাজটাই করে।

Key Terms (সহজ English definition):

Non-parametric — এই algorithm কোনো fixed formula বা assumption ধরে নেয় না। Data যেভাবে আছে, সেভাবেই কাজ করে।

Instance-based / Lazy Learning — KNN কোনো "training" করে না। সে সব training data মনে রেখে দেয়, আর prediction এর সময় সেখান থেকে খোঁজে। তাই একে "lazy" বলে — কারণ সে আগে কোনো কাজ করে না।

Classification — input দেখে বলে "এটা কোন category তে পড়ে?" যেমন: এই ফুলটা কি Rose নাকি Lily?

Regression — input দেখে একটা সংখ্যা predict করে। যেমন: এই বাড়ির দাম কত হবে?

KNN এর মূল idea টা কী?

"তুমি যার কাছাকাছি, তুমি তার মতোই।"
(You are like your nearest neighbors.)

একটা নতুন data point আসলে KNN বলে:

- এই নতুন point এর সবচেয়ে কাছের **k** টা পুরনো point খোঁজো
 - সেই k টা point এর মধ্যে যে **label** বেশি — সেটাই হবে নতুন point এর label
-

KNN এ কোনো Training Phase নেই — এর মানে কী?

সাধারণ ML algorithms (যেমন Linear Regression) training এর সময় একটা formula বা equation তৈরি করে নেয়। পরে prediction এ সেই formula ব্যবহার করে।

কিন্তু KNN তা করে না। সে শুধু সব training data টা "মুখস্থ" করে রাখে। Prediction এর সময় সরাসরি সেই data থেকে calculate করে। এই কারণে:

-  Training time = ০ (কোনো training লাগে না)
 -  Prediction time = বেশি (কারণ প্রতিবার সব data থেকে calculate করতে হয়)
-

২. Algorithm Steps — KNN কীভাবে কাজ করে?

ধরো তোমার কাছে কিছু পুরনো data আছে (training data), আর একটা নতুন point এসেছে যেটাকে classify করতে হবে।

Step 1: k এর মান ঠিক করো

k মানে হলো — "আমি কতজন প্রতিবেশীকে দেখব?"

যদি k = 3 হয়, তাহলে নতুন point এর সবচেয়ে কাছের ৩টা point দেখব।

Step 2: সব training point এর সাথে distance calculate করো

নতুন point Q আর প্রতিটা training point এর মধ্যে দূরত্ব (distance) বের করো। (কীভাবে distance বের করে — সেটা নিচে Distance Metrics section এ বিস্তারিত আছে।)

Step 3: সবচেয়ে কাছের k টা point বেছে নাও

Distance অনুযায়ী sort করে সবচেয়ে ছোট distance এর k টা point নাও।

Step 4: সিদ্ধান্ত নাও

- **Classification** হলে → **Majority Vote** মানে, k জন প্রতিবেশীর মধ্যে যে label সবচেয়ে বেশিবার আছে, সেটাই হবে answer।
- **Regression** হলে → **Average** মানে, k জন প্রতিবেশীর values এর গড় (average) নাও।

৩. Distance Metrics — দূরত্ব কীভাবে মাপা হয়?

এটা KNN এর সবচেয়ে গুরুত্বপূর্ণ অংশ। কারণ "কাছের" মানে নির্ভর করে তুমি দূরত্ব কীভাবে মাপছো তার উপর।

● Euclidean Distance (সবচেয়ে বেশি ব্যবহৃত)

সহজ definition: দুটো point এর মধ্যে সরাসরি সোজা রেখার দূরত্ব। এটাই আমরা স্কুলে পড়া "পিথাগোরাসের থিওরেম" দিয়ে বের করি।

Formula (2D এর জন্য):

$$d = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$$

Formula (n dimension এর জন্য):

$$d = \sqrt{\sum_{i=1}^n (x_i - y_i)^2}$$

উদাহরণ: Point A = (1, 2), Point B = (4, 6)

$$d = \sqrt{(4 - 1)^2 + (6 - 2)^2} = \sqrt{9 + 16} = \sqrt{25} = 5$$

● Manhattan Distance

সহজ definition: দুটো point এর মধ্যে দূরত্ব যদি তুমি শুধু আনুভূমিক (horizontal) আর উল্লম্ব (vertical) — এই দুই দিকে চলতে পারো। যেমন শহরের রাস্তায় চলার মতো — তুমি কোনাকুনি যেতে পারবে না।

Formula:

$$d = \sum_{i=1}^n |x_i - y_i|$$

উদাহরণ: Point A = (1, 2), Point B = (4, 6)

$$d = |4 - 1| + |6 - 2| = 3 + 4 = 7$$

● Minkowski Distance (সবার generalized version)

সহজ definition: এটা একটা "master formula" যেটা দিয়ে Euclidean আর Manhattan দুটোই বের করা যায়, শুধু p এর মান বদলে।

Formula:

$$d = \left(\sum_{i=1}^n |x_i - y_i|^p \right)^{1/p}$$

- যদি $p = 1$ হয় → Manhattan Distance
- যদি $p = 2$ হয় → Euclidean Distance

তিনটার তুলনা এক জায়গায়:

Distance	কীভাবে চলে	কখন ভালো
Euclidean	সোজা রেখায়	সাধারণ continuous data তে
Manhattan	শুধু আনুভূমিক/উল্লম্ব	High-dimensional data তে অনেক সময় ভালো
Minkowski	p দিয়ে control করা যায়	Flexible, p tune করা যায়

8. k কীভাবে choose করবে?

এটা KNN এর সবচেয়ে বড় সিদ্ধান্ত।

k ছোট হলে কী হয়? (যেমন $k=1$)

শুধু ১ জন প্রতিবেশী দেখছে।

- সমস্যা: যদি সেই ১টা point টা noise বা ভুল data হয়, তাহলে পুরো prediction ভুল হবে।
- এটাকে বলে **Overfitting** — model training data তে খুব ভালো করে, কিন্তু নতুন data তে ব্যর্থ হয়।

k বড় হলে কী হয়? (যেমন $k=100$)

অনেক প্রতিবেশী দেখছে।

- সমস্যা: অনেক দূরের point গুলোও vote দিচ্ছে, যেগুলো আসলে relevant না।
- এটাকে বলে **Underfitting** — model সব কিছুকে একই রকম ভাবে দেখছে, কোনো pattern ধরতে পারছে না।

তাহলে সঠিক **k** কীভাবে বেরবে?

Cross-validation ব্যবহার করো।

মানে, বিভিন্ন **k** এর মান দিয়ে test করো এবং দেখো কোন **k** তে সবচেয়ে ভালো accuracy পাওয়া যাচ্ছে।

একটা practical টিপস: সাধারণত $k = \sqrt{n}$ (n = training data এর সংখ্যা) দিয়ে শুরু করা হয়।

Feature Scaling কেন জরুরি?

ধরো তোমার data তে দুটো feature আছে:

- বয়স: 25, 30, 35 (ছোট সংখ্যা)
- বেতন: 50000, 60000, 70000 (বড় সংখ্যা)

Distance calculate করার সময় বেতনের পার্থক্য অনেক বেশি বড় দেখাবে, ফলে বয়স প্রায় কোনো ভূমিকাই রাখবে না!

তাই সব feature কে একই scale এ নিয়ে আসতে হয় — এটাকে বলে **Normalization** বা **Standardization**।

৫. Pros and Cons

✓ সুবিধা:

- **Simple and intuitive** — বোঝা সহজ, implement করা সহজ
- **No training time** — data দেখলেই কাজ শুরু করতে পারে
- **Adapts to new data** — নতুন data যোগ হলে কোনো retraining লাগে না

✗ অসুবিধা:

- **Slow prediction** — প্রতিবার সব training data থেকে distance calculate করতে হয়
- **Sensitive to irrelevant features** — অপ্রয়োজনীয় feature থাকলে result খারাপ হয়
- **Curse of Dimensionality** — feature সংখ্যা অনেক বেশি হলে "কাছের" আর "দূরের" পার্থক্য করা কঠিন হয়ে পড়ে

Curse of Dimensionality মানে কী? যখন data এর dimensions (features) অনেক বেশি হয়, তখন সব point গুলো একে অপর থেকে প্রায় সমান দূরত্বে থাকে। ফলে "nearest neighbor" এর concept টা আর কাজ করে না।

৬. Math Example — সম্পূর্ণ Step by Step

এখন document এর math example টা একদম বিস্তারিতভাবে করি।

Dataset:

Point	x	y	Label
A	1	2	Red
B	2	3	Red
C	3	3	Blue
D	6	5	Blue

Query Point: Q = (3, 4), k = 3

মানে, আমরা Q কে classify করতে চাই — এটা Red নাকি Blue?

Step 1: Q থেকে প্রতিটা point এর Euclidean Distance বের করো

Formula: $d = \sqrt{(x_Q - x)^2 + (y_Q - y)^2}$

d(Q, A): $d = \sqrt{(3-1)^2 + (4-2)^2} = \sqrt{2^2 + 2^2} = \sqrt{4+4} = \sqrt{8} \approx 2.83$

d(Q, B): $d = \sqrt{(3-2)^2 + (4-3)^2} = \sqrt{1^2 + 1^2} = \sqrt{1+1} = \sqrt{2} \approx 1.41$

d(Q, C): $d = \sqrt{(3-3)^2 + (4-3)^2} = \sqrt{0^2 + 1^2} = \sqrt{0+1} = \sqrt{1} = 1.0$

d(Q, D): $d = \sqrt{(3-6)^2 + (4-5)^2} = \sqrt{(-3)^2 + (-1)^2} = \sqrt{9+1} = \sqrt{10} \approx 3.16$

Step 2: Distance অনুযায়ী sort করো (ছোট থেকে বড়)

Point	Distance	Label
C	1.0	Blue
B	1.41	Red
A	2.83	Red
D	3.16	Blue

Step 3: $k = 3$, তাই সবচেয়ে কাছের ৩টা নাও

✓ C → Blue ✓ B → Red ✓ A → Red

(D বাদ, কারণ সে 4th)

Step 4: Majority Vote

- Red: 2 votes (B, A)
- Blue: 1 vote (C)

🎯 **Final Answer: Q কে Red classify করা হলো!**

এই example টা visually বোঝা:

```
y
5 |      D(Blue)
4 |      Q(?)
3 |  B(Red) C(Blue)
2 |  A(Red)
1 |
+-----> x
1 2 3 4 5 6
```

Q = (3,4) এর সবচেয়ে কাছে আছে C, B, A। এদের মধ্যে Red বেশি, তাই Q = Red।

সারসংক্ষেপ (Summary)

বিষয়	মূল কথা
KNN কী	Lazy, non-parametric algorithm
কাজ কী	Classification ও Regression
Distance	Euclidean সবচেয়ে common
k ছোট	Overfitting এর ঝুঁকি
k বড়	Underfitting এর ঝুঁকি

Feature
Scaling

অবশ্যই করতে হবে

দুর্বলতা

Slow prediction, Curse of Dimensionality
